

# Hlohovec Space Program

Everybody, put your hands together for Flatland's first astronaut! Here is your spaceship, and here is your mission briefing.

Your spaceship is, obviously, a point in a flat two-dimensional plane. We will start with just space maneuvers without external gravity.

The state of your spaceship at any given time is given by a two-dimensional point and a two-dimensional vector: its position and its velocity. Time is discrete – we use nonnegative integers to refer to moments in time. Your starting time is 0. Your flight will therefore correspond to a sequence of positions  $p_0, p_1, \dots$  and a sequence of velocities  $v_0, v_1, \dots$ . In each moment, the next position  $p_{t+1}$  and velocity  $v_{t+1}$  of your spaceship are determined based on its previous position  $p_t$ , its previous velocity  $v_t$ , and the acceleration vector (pilot input)  $a_t$  chosen in the previous moment, as follows:

$$v_{t+1} = v_t + a_t$$

$$p_{t+1} = p_t + v_{t+1}$$

The cost incurred between moments  $t$  and  $t + 1$  is given by

$$\max(\|a_t\| - a_{\text{free}}, 0)^{1.5}$$

where  $\|x\|$  denotes the Euclidean norm of  $x$  (i.e., the length of the vector  $x$ ) and  $a_{\text{free}}$  is a given *free acceleration threshold* – a value that specifies how much acceleration we can exert for free in each time step.

The total cost of your flight is the sum of the costs of its individual steps.

## Task

What is the mission, you ask? Each mission is given by:

- the duration of the mission – the exact number  $t$  of time steps it should take (you can't be late, but you also can't be early),
- the free acceleration threshold  $a_{\text{free}}$ .
- the mission's initial state: the position  $p_0$  and the incoming velocity  $v_0$
- the mission's target state: the position  $p_t$  after  $t$  steps and the **next** desired velocity  $v_{t+1}$

Your task is to find a way to fly the mission in a way that's as cheap as possible.

## Input format

The first line of each input file contains the number  $T$  of test cases. The specified number of test cases follows, one after another.

Each test case begins with an empty line followed by three lines describing the test case. Each of these three lines consists of multiple space-separated numbers:

- The first line contains  $t$  and  $a_{\text{free}}$ : the exact length of the mission and the free acceleration threshold.
- The second line contains the initial position  $x_0, y_0$  and the initial velocity  $v_{0,x}, v_{0,y}$
- The third line contains the final position  $x_t, y_t$  and the post-final velocity  $v_{t+1,x}, v_{t+1,y}$

The duration of the mission  $t$  is positive integer. All given position coordinates are real numbers from  $-10^4$  to  $10^4$ . All given velocity components are real numbers from  $-500$  to  $500$ . The free acceleration threshold is a nonnegative real number not exceeding  $500$ .

(These constraints only apply to the input. Your solution can visit points at other coordinates and have a velocity outside of the given range.)

## Output format

For each test case, output  $t - 1$  lines. The  $i - th$  line, for  $i = 1..(t - 1)$ , should contain the position  $p_i$  of the spaceship at time  $i$ , i.e., its coordinates  $x_i, y_i$  separated by space. Note that this means that you should omit the first and last positions of the ship (which are fixed). Print an empty line after each test case, including the last one.

Note that this fully determines your flight:

- The full sequence of positions  $p_0, \dots, p_t$  of your spaceship determines its velocity vectors  $v_1, \dots, v_t$ .
- Together with the given  $v_0$  and  $v_{t+1}$  from the input, this second sequence then determines the accelerations  $a_0, \dots, a_{t-1}, a_t$ .
- Note that  $a_t$  is the acceleration you have to choose in time  $t$ , i.e., at the moment you reach the given target position, to adjust your speed to match the given target velocity.
- The  $t + 1$  values  $a_i$  are used to score your solution.

(We recommend rounding the output to about 9 decimal places. Don't accidentally exceed maximum output size by printing way too many unnecessary decimals.)

## Scoring

For each individual test case, let  $x$  be your total cost + 0.1, and let  $y$  be (1.01 times the cost of our best solution) + 0.1. The badness of this test case will be  $\max(1, x/y)$ .

For a subproblem:

- let  $a$  be the median and  $b$  the maximum of the badnesses of the individual test cases,
- let  $a' = \min(a, 10)$  and  $b' = \min(b, 10)$ , and
- let  $m$  be the maximum number of points for that subproblem.

Then your score for that subproblem will be:

$$m \times \left( 2^{-a'} + 2^{-b'} \right)$$

For each subproblem your final score will be rounded to two decimal places.

## Special rule

Resubmissions for this problem do not generate penalty minutes.

### Subproblem H1 (20 points, public)

Input file: [H1.in \(/static/online2026/H1.f3fb96d7f6eb88b6463bdcda62eb610f.in\)](/static/online2026/H1.f3fb96d7f6eb88b6463bdcda62eb610f.in).

Constraints:  $1 \leq T \leq 100$  and in each test case  $t \leq 100$  and the free acceleration is zero ( $a_{\text{free}} = 0$ ).

### Subproblem H2 (40 points, private)

Input file: [H2.in \(/static/online2026/H2.98b8cbcd5b67b65d15037bbe39e7d89a.in\)](/static/online2026/H2.98b8cbcd5b67b65d15037bbe39e7d89a.in).

Constraints:  $1 \leq T \leq 100$  and in each test case  $t \leq 20$ .

### Subproblem H3 (40 points, private)

Input file: [H3.in \(/static/online2026/H3.d0c56e1c89163470a3c103d60fc0e19b.in\)](/static/online2026/H3.d0c56e1c89163470a3c103d60fc0e19b.in).

Constraints:  $1 \leq T \leq 100$  and in each test case  $t \leq 500$ .

## Example

**input**

2

11 1.0

0.0 0.0 -5.0 1.0

20.0 0.0 0.0 0.0

15 1.0

0.0 0.0 -5.0 5.0

20.0 0.0 -5.0 5.0

**output**

-1.01983965 1.11453243

-1.04617716 2.34652886

-0.07968654 3.73208262

1.85994827 4.92023509

4.68340046 5.63985689

7.86025174 5.43857250

11.45944848 4.42659305

14.94703247 2.84834759

17.50567745 1.64659492

19.15956984 0.53233207

-0.83173191 2.21460671

-0.83693174 3.86498242

0.09968395 5.18075863

1.88417559 5.96169037

4.09007635 5.83907414

6.70114658 4.80157066

9.08823359 2.79061024

11.47457156 0.52624337

13.54036144 -1.85930880

16.05223081 -3.45047051

18.23456293 -4.11947032

19.90269709 -3.92729193

20.82050556 -3.07957015

20.82522058 -1.82175055

The outputs shown in the example output section are not the optimal solutions for these test cases.